Actualidad Huancayo

Portada

Universidad Continental

Curso: Pruebas y Calidad de Software

Producto Académico N.º 2: Blog de noticias autoadministrable en PHP, plan de pruebas e informe del código fuente

Autor: Ramos Ore Dennis Jhordan

Modalidad: individual

Fecha del registro de comprobaciones: 13 de octubre de 2026, America/Lima. No se conoce fecha de entrega.

Introducción

Actualidad Huancayo es un proyecto académico que relaciona construcción de una aplicación web con planificación y ejecución de pruebas. El desarrollo permite publicar noticias desde un panel y consultar únicamente las publicadas desde el área pública. La solución utiliza PHP 8.2, PDO y MariaDB/MySQL sin framework, con recursos locales para facilitar su instalación y explicación.

La consigna identifica la actividad como individual aunque contiene referencias a un equipo. Se conserva un único autor, Ramos Ore Dennis Jhordan, sin inventar integrantes. Los textos de demostración son originales y ficticios; no deben presentarse como noticias verificadas. Los materiales de las semanas 3 y 4 no se proporcionaron y deben contrastarse antes de la entrega.

Problema y objetivos

Actualizar una página mediante edición directa de archivos requiere conocimientos técnicos y favorece errores. Un panel permite gestionar publicaciones, categorías y estados con validaciones consistentes. La aplicación también debe impedir que un visitante consulte un borrador, que una categoría eliminada arrastre sus noticias o que entradas externas se conviertan en código SQL/HTML ejecutable.

El objetivo general es construir un blog funcional, sencillo y explicable, acompañado de evidencia de calidad. Los objetivos específicos son separar responsabilidades con MVC, implementar administración segura, facilitar consulta por filtros y paginación, ofrecer interacción clara y adaptable y documentar tanto las pruebas realizadas como sus límites.

Alcance y requisitos

Incluye inicio, detalle, búsqueda por título, filtro por categoría, paginación y estado sin resultados. El panel contiene login/logout, resumen, consulta y CRUD de noticias y categorías. Las noticias tienen título, resumen, contenido textual, categoría, imagen opcional, texto alternativo, estado y fechas.

Se identificaron diez requisitos funcionales RF01–RF10 y ocho no funcionales RNF01–RNF08, definidos en el plan y vinculados en la matriz. Los principales no funcionales cubren PDO, CSRF, XSS, imágenes, accesibilidad parcial, adaptación visual, eficiencia e instalación/aislamiento. No incluye comentarios, registro público, recuperación de contraseña, múltiples roles ni servicios externos.

Arquitectura y estructura

MVC organiza responsabilidades sin añadir dependencias: el controlador decide la acción, el modelo consulta y modifica datos y las vistas producen HTML. La aplicación tiene un único punto de entrada público.

```
app/
  bootstrap.php                Carga de clases y conexión PDO
  support.php                  Escape, URL, CSRF y validación
  Controllers/Controller.php   Rutas, autorización y acciones
  Models/Blog.php              Consultas y persistencia
  Views/                       Plantillas públicas y administrativas
config/config.example.php      Configuración sin secretos locales
database/schema.sql            Estructura y restricciones
database/demo.sql              Noticias originales ficticias
public/index.php               Punto de entrada
public/assets/                 CSS y JavaScript locales
public/uploads/                Imágenes y protección Apache
scripts/                       Creación del administrador y datos de prueba
tests/                         Unitarias, integración, HTTP, navegador, rendimiento
docs/                          Documentación y evidencias
storage/                       Sesiones y datos temporales, excluidos
delivery/                      ZIP final, excluido del repositorio
```

Diseño de la base de datos

Tabla	Campos esenciales	Integridad
admins	id, email, password_hash, created_at	Email único; hash VARCHAR(255).
categories	id, name	Nombre único, hasta 60 caracteres.
news	id, title, summary, content, category_id, image, image_alt, status, published_at, created_at, updated_at	Categoría obligatoria; estado draft/published; FK RESTRICT.

Una categoría puede tener muchas noticias; cada noticia pertenece a una categoría. El administrador representa la identidad que accede al panel; no se añadió relación de autoría múltiple porque no está solicitada. La clave foránea de news.category_id impide eliminar una categoría con noticias. Para retirarla es necesario reasignar noticias o eliminarlas individualmente con confirmación. No hay borrado en cascada.

La codificación utf8mb4 permite textos españoles. Los índices de estado/fecha/ID y categoría/estado/fecha apoyan selección pública y filtros. La búsqueda contiene LIKE con comodines a ambos extremos: sirve para el volumen académico, pero no se afirma que esos índices aceleren toda búsqueda de texto. Si el volumen creciera, habría que medir y evaluar búsqueda especializada.

created_at registra creación; updated_at cambia al editar. Publicar asigna published_at, editar una publicada conserva esa fecha y regresar a borrador la limpia. PHP y la conexión MySQL se configuran

para Lima (UTC-5), para presentar las fechas en el mismo huso. La ordenación pública utiliza published_at descendente con ID como desempate.

Archivos principales y flujo de petición

- El navegador solicita `public/index.php?r=home` u otra ruta. El punto de entrada configura cookies/sesión y cabeceras, carga configuración local y crea la conexión PDO.
- `Controller::run` valida la ruta y el método. Antes de cualquier acción del panel comprueba la sesión; todo POST exige CSRF. Rutas públicas de lectura rechazan POST.
- Para un listado, `Blog::listing` construye condiciones a partir de fragmentos SQL internos y enlaza valores externos. Cuenta resultados y aplica LIMIT 6/OFFSET en la base; ajusta páginas fuera de rango. Los enlaces conservan q y categoría.
- El controlador entrega datos a la vista. `layout.php` aporta cabecera, navegación, contenido y pie. Cada valor externo mostrado en HTML utiliza `e`, que llama a htmlspecialchars con UTF-8.
- Un POST válido guarda y redirige para evitar repetición accidental al recargar. Si falla la validación, se vuelve a representar el formulario con textos y errores asociados. El selector de archivo debe elegirse otra vez porque el navegador no conserva archivos después de una respuesta.

`create-admin.php` solo se ejecuta por CLI, pide la contraseña mediante stdin y usa password_hash; no reemplaza cuentas existentes. `setup-test.php` es un helper de bases `_test`, no una instalación de desarrollo. `unit.php` prueba funciones, `integration.php` el modelo y restricciones reales, `http.php` peticiones y cookies, `browser.mjs` interacción/capturas y `performance.php` el escenario de volumen.

Validaciones y seguridad

Título de 5 a 160 caracteres, resumen de 10 a 300 y contenido de 20 a 20.000, calculados con mb_strlen después de trim. Categoría debe ser entero positivo y existir. Estado solo admite draft/published. Nombre de categoría de 2 a 60 y único. Con imagen se requiere alt de 1 a 180; sin imagen el campo puede estar vacío. Los errores aparecen junto al campo y aria-describedby relaciona el control con su mensaje; el primer error recibe foco mediante JavaScript sencillo.

PDO utiliza consultas preparadas con emulación desactivada. Los nombres de tablas para eliminación provienen únicamente de una elección interna; nunca se incorporan nombres SQL enviados por usuario. La búsqueda escapa los comodines %, _ y el carácter de escape para tratarlos literalmente. Las contraseñas se comprueban con password_verify y el login regenera el ID de sesión y el token CSRF.

Las cookies son HttpOnly, SameSite=Lax y Secure cuando se utiliza HTTPS. Los formularios que modifican datos exigen POST y un token aleatorio comparado mediante hash_equals. El contenido de noticia es texto plano escapado, sin aceptar HTML. Se agregan CSP, X-Content-Type-Options y Referrer-Policy para reducir exposición del navegador; estas cabeceras no sustituyen la validación del servidor.

Las imágenes se revisan por error de recepción, tamaño hasta 2 MiB, extensión, MIME real con fileinfo y reconocimiento con getimagesize. Solo se aceptan JPEG/PNG/WebP. El servidor genera nombre aleatorio hexadecimal y usa move_uploaded_file. Apache dispone de .htaccess que niega extensiones ejecutables y desactiva índices/CGI. Se comprobó rechazo de archivos falsos y de extensión PHP por la aplicación; Apache también denegó con 403 un PHP colocado como fixture de prueba en uploads. Las imágenes antiguas se conservan al reemplazar o eliminar una noticia para evitar borrado accidental: una limpieza futura debe verificar referencias.

La configuración local y de pruebas se excluye del repositorio y ZIP. Solo public debe ser la raíz servida. Errores de conexión producen 503 con explicación genérica; el detalle técnico va al registro. El alcance es académico local: no se implementó limitación de intentos, recuperación de cuentas ni infraestructura de producción.

Usabilidad, accesibilidad, estilo y eficiencia

La interfaz está en español, presenta mensajes concretos, permite limpiar filtros y exige confirmación en una pantalla antes de eliminar. Los borradores y publicados tienen texto identificador; no dependen solo de color. Formularios mantienen valores cuando hay errores y señalan cómo corregirlos.

HTML incluye header, nav, main, article y footer, etiquetas asociadas, enlace de salto al contenido, foco visible y errores por campo. No hay animaciones. Las imágenes requieren alt y las de listados usan loading=lazy. Se comprobó salto con teclado, etiquetas de login y foco de errores; aún falta revisión completa con lector de pantalla y medición sistemática de contraste. No se afirma conformidad total con WCAG.

El estilo emplea naranja suave, beige, blanco y café; tipografía de sistema y encabezados serif locales. Las tarjetas pasan de tres a dos y una columna según ancho; los filtros se apilan en celular. Se revisaron capturas en 1440×1000 y 390×844 emulados, sin desbordamiento. Otros dispositivos, zoom y navegadores quedan pendientes.

La paginación ocurre en SQL, no cargando todos los registros en PHP. Los recursos son locales y no requieren framework ni compilación. En un escenario local con 1.000 noticias adicionales se midió mediana de 9,946 ms, P95 de 35,225 ms y máximo de 41,897 ms para 30 solicitudes HTTP secuenciales a página 100 tras un calentamiento. Es evidencia del escenario descrito y no de carga concurrente o red externa.

Resumen del plan y resultados

El plan diseña 24 casos de alto valor vinculados a 18 requisitos, con particiones, límites y transiciones. Priorización P1 para confidencialidad, pérdida de datos y operaciones críticas. Incluye criterios de entrada/salida/suspensión, cronograma relativo y estimación de 22 horas de trabajo de pruebas, no tiempo medido.

Se ejecutaron 29 comprobaciones unitarias, 14 de integración MariaDB, 44 HTTP, 11 en Edge y siete de instalación desde ZIP con Apache 2.4.58: 105 comprobaciones aprobadas sin fallos finales. Los 24 casos del plan tienen 23 Aprobado y uno No ejecutado, que no deben confundirse con las 105

aserciones. Se ejecutó además sintaxis PHP y el escenario de rendimiento. Los comandos y registros reales están en resultados-de-pruebas.md y evidencias.

La base de integración realiza rollback; la de HTTP contiene exclusivamente demostración y datos temporales. Se utilizó una instancia MariaDB nueva en storage/mysql, puerto 3307, sin modificar servicios ni bases de XAMPP. Los datos de desarrollo del receptor no se han probado ni eliminado.

Limitaciones y conclusiones

La solución implementa los flujos esenciales y controles solicitados en un tamaño explicable. Las pruebas ejecutadas respaldan reglas y operaciones concretas; no garantizan ausencia de todos los defectos. CP21 (accesibilidad manual completa) permanece No ejecutado. Todos los P1 están aprobados, incluido CP24 con instalación en una copia extraída desde ZIP, importación SQL y administrador creado por CLI. La aceptación del pendiente P2 corresponde al autor. Quedan lector de pantalla, contraste sistemático, otros navegadores/dispositivos, JPEG/WebP visuales y carga concurrente fuera de la ejecución realizada.

Antes de presentar, el autor debe instalar desde ZIP, revisar esos casos, contrastar semanas 3 y 4 y comprobar que puede explicar el flujo y las decisiones. El valor académico consiste tanto en el blog como en distinguir requisitos, pruebas planificadas, comprobaciones ejecutadas y límites de la evidencia.

Referencias verificables

Fuentes técnicas oficiales consultadas para documentar mecanismos; no se atribuyen materiales del curso no recibidos.

- PHP, password_hash: API de hash; el proyecto usa PASSWORD_DEFAULT con PHP 8.2. No asume que valores predeterminados de versiones posteriores sean iguales.
- PHP, consultas preparadas PDO: separación entre sentencia y parámetros de datos.
- W3C WAI, notificaciones de formularios: comunicación de errores y asociación con controles. Consultar esta guía no equivale a certificar accesibilidad.

Anexos

A: plan-de-pruebas.md y matriz-trazabilidad.md. B: resultados-de-pruebas.md y evidencias textuales. C: README y SQL. D: guion-de-exposicion.md y participantes.md.

Capturas reales: inicio-escritorio.png, inicio-celular.png, login-celular.png, formulario-celular.png, errores-celular.png y panel-escritorio.png en evidencias. Fueron obtenidas con Edge, no son maquetas generadas.

Espacio pendiente A: captura adicional de la importación mediante interfaz phpMyAdmin; no obtenida (la importación SQL real se verificó mediante PDO). **Espacio pendiente B:** evidencia del recorrido completo con lector de pantalla; no obtenida. Adjuntar solo después de ejecutar y registrar resultado

real. La comprobación Apache de denegación de scripts en uploads y credenciales fuera de la raíz pública ya tiene evidencia textual en install.txt.